Administración de Sistemas Operativos

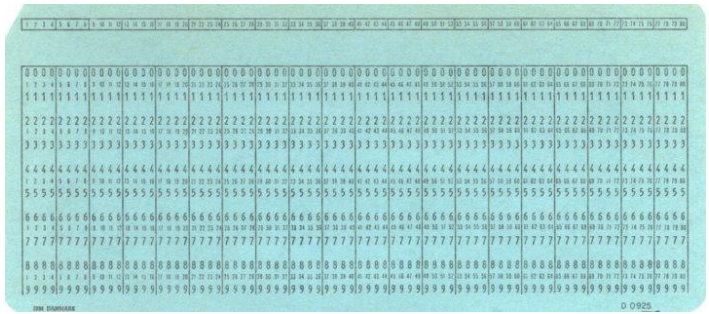
David Ubide Alaiz



Por qué planificar tareas

El concepto de procesamiento por lotes es fundamental y antecede incluso a las computadoras modernas.

Tarjetas Perforadas: Su origen se relaciona directamente con la época de las tarjetas perforadas (décadas de 1950 y 1960). Como el acceso a los mainframes era caro y limitado, los programadores agrupaban sus trabajos (lotes de tarjetas con datos e instrucciones) y los enviaban para ser procesados por la computadora sin interacción en tiempo real.



Por qué planificar tareas

El concepto de procesamiento por lotes es fundamental y antecede incluso a las computadoras modernas.

Sistemas Operativos: Los primeros sistemas operativos por lotes (como el IBM OS/360) gestionaban colas de estos trabajos. Esto permitía la optimización de los costosos recursos informáticos, asegurando que la CPU estuviera siempre ocupada.



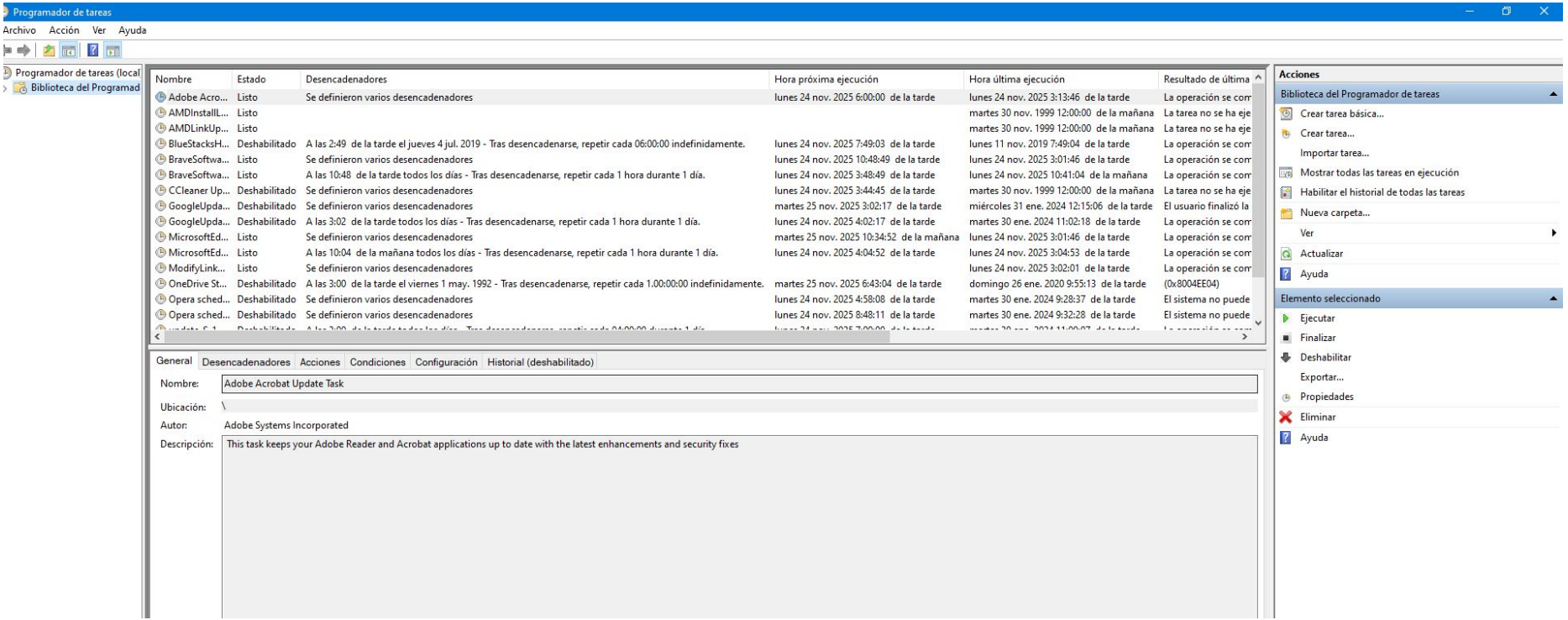
Por qué planificar tareas

El concepto de procesamiento por lotes es fundamental y antecede incluso a las computadoras modernas.

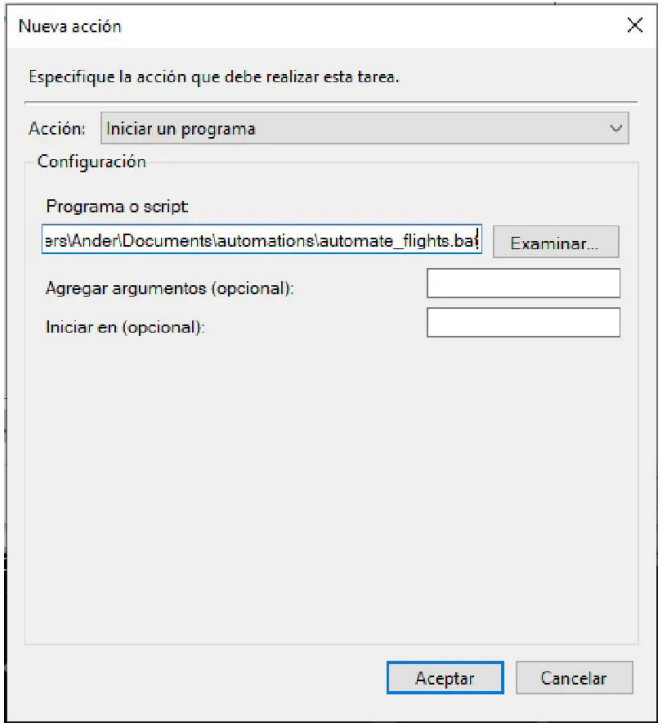
Archivos Batch: En el contexto de sistemas operativos personales como MS-DOS (a partir de 1980), la programación por lotes se materializó en los archivos .bat. Estos son scripts de texto plano que contienen una serie de comandos de la línea de comandos que se ejecutan secuencialmente (en lote). El término batch (lote) en la extensión .bat proviene de este concepto.



¿Y si quiero ejecutar un programa?



¿Y si quiero ejecutar un programa?



[Automatizar script de Python en Windows y Mac - Ander Fernández]

Índice

- Ejecución programada: comando at
- Ejecución periódica:
  - Comando cron
  - Comando anacron

Introducción

En esta sección veremos la utilización de comandos que permiten planificar tareas para ejecutar en un futuro:

- at, batch: permiten ejecutar trabajos a una hora específica o bajo determinadas condiciones.
- cron (y anacron) permite correr trabajos a intervalos regulares

Estos comandos permiten automatizar las distintas tareas, para que se ejecuten en determinados momentos sin intervención del administrador.

Introducción

- ➔ Realizar copias de seguridad automáticas
- ➔ Ejecutar scripts de mantenimiento o limpieza de archivos
- ➔ Enviar correos o reportes periódicos
- ➔ Sincronización de datos o servidores
- ➔ Reiniciar servicios o aplicaciones en horarios definidos

Comando at

Permite indicar el momento en que se quiere ejecutar un trabajo

Sintaxis:

    at [opciones] TIME

Al ejecutar at pasamos a un nuevo prompt, que nos permite introducir comandos que se ejecutarán a la hora indicada

- para salvar el trabajo y salir CTRL-D
- el entorno actual también se salva
- al terminar, la salida estándar se envía como un mail al usuario
- el trabajo no se para al salir de la sesión

Comando at

Ejemplo:

$ at 11:45
warning: commands will be executed using /bin/sh
at> ls /tmp > lista
at> <EOT>
job 4 at Wed Nov 16 11:45:00 2005

Algunas opciones:

- -f FILE especifica un fichero conteniendo las acciones a realizar en vez de la entrada estándar
- -c jobnumber muestra el trabajo a ejecutar
- -m envía un mail al usuario, incluso aunque no haya salida
- -v muestra la hora a la que se ejecutará el trabajo

TIME puede especificarse de varias formas:

- HH:MM por ejemplo 12:54
- HH:MMAM/PM, por ejemplo 1:35PM
- HH:MM MMDDYY, por ejemplo 1:35PM 122505
- now + numero unidades , donde unidades puede ser minutes, hours, days, o weeks

  $ at now+2hours
- today, tomorrow, por ejemplo 12:44tomorrow
- midnight (00:00), noon (12:00), teatime (16:00)

Comandos Relacionados

- atq o at -l lista los trabajos pendientes del usuario
  - si es root, lista los trabajos de todos los usuarios
- atrm o at -d borra trabajos identificados por su número de trabajo
- batch ejecuta trabajos cuando la carga del sistema es baja
  - el trabajo empieza en cuanto la carga caiga por debajo de 1.5
  - la carga se obtiene del fichero /proc/loadavg

AT: Ficheros de Configuración

El administrador puede controlar la utilización de at

- Ficheros /etc/at.allow y /etc/at.deny
  - at.allow lista los usuarios que pueden usar at
  - at.deny lista los usuarios que NO pueden usar at
- Primero se chequea /etc/at.allow
  - si está el usuario, puede usar at
  - si no está o el fichero no existe, se chequea /etc/at.deny
  - si el usuario no está en at.deny puede usar at
- Si no existe ninguno de los dos ficheros, solo root puede ejecutar at
- Para dar permiso para todos los usuarios crear sólo el fichero at.deny vacío

Procesos Periódicos: cron

Para crear trabajos que se ejecuten periódicamente se utilizan el daemon cron y el comando crontab

- crontab permite configurar los procesos periódicos
- cron se encarga de su ejecución

La utilización de cron se gestiona a través de los ficheros /etc/cron.allow y /etc/cron.deny

- el comportamiento si no existen los ficheros depende de la configuración del sistema
- en Debian, si no existen, todos los usuarios pueden usar crontab

Fichero crontab

Los trabajos se especifican en un fichero de crontab, que se guarda en /var/spool/cron/crontabs, y que puede tener tres tipos de líneas:

- Comentarios, que empiezan por #
  Definición de variables, de tipo nombre = valor
  # shell usada para ejecutar los comandos
  SHELL=/bin/bash
  # Usuario al que se envía (por mail) la salida
  # de los comandos (por defecto, se envían
  # al propietario del fichero crontab)
  MAILTO=pepe

- Especificación del trabajo y de la hora de ejecución, de la siguiente forma:

  minuto hora día mes día semana comando

  - el día de la semana de 0 a 7 (0 ó 7 domingo)
  - * indica cualquier valor
  - se pueden indicar rangos, listas o repeticiones:
  - 1-5 para indicar de lunes a viernes
  - 0,15,30,45 para indicar cada 15 minutos
  - 0-23/2 en el campo hora, indica realizar cada dos horas (0, 2, 4, etc.)

Ejemplos:

- Borra el /tmp todos los días laborables a las 4:30 am:
  30 4 * * 1-5 rm -rf /tmp/*
- Escribe la hora, cada 15 minutos, durante la noche:
  0,15,30,45 0-8,20-23 * * * echo Hora:$(date)>>/tmp/horas



https://crontab.guru/ Validamos que esté bien

Comando crontab

Para crear y editar los trabajos periódicos se puede utilizar el comando crontab

Sintaxis:

crontab [-u usuario] {-l|-e|-r}
crontab [-u usuario] fichero

- en la segunda forma instala un nuevo crontab desde un fichero

Opciones:

- -u usuario crea o maneja el crontab de un usuario específico (sólo root)
- -e edita el fichero crontab
- -l muestra el fichero crontab
- -r borra el fichero crontab

Fichero /ect/crontab y /etc/cron.d

Cron busca ficheros en /var/spool/cron para ejecutarlos a la hora indicada

- además también ejecuta las acciones indicadas en los ficheros /etc/crontab y en el directorio /etc/cron.d/
- estos ficheros suelen ser de mantenimiento del sistema

De esta forma, el administrador puede crear scripts que se ejecuten con periodicidad horaria, diaria, semanal y mensual.

- solo tiene que colocar estos scripts en los directorios /etc/cron.hourly, /etc/cron.daily, /etc/cron.weekly o /etc/cron.monthly
- la fecha y hora de ejecución de estos scripts se controla en el fichero /etc/crontab

Fichero etc/crontab (debian)

Si necesitas características más avanzadas de las que ofrecen los cron jobs te recomendamos consultar las posibilidades de los systemd timers. Algunas características avanzadas que nos ofrecen como extra los systemd timers son estas:

- ➔ Integración nativa con el demonio systemd.
- ➔ Posibilidad de ejecutar tareas según condiciones del sistema (por ejemplo, al arrancar o al conectar la red).
- ➔ Retrasos y tolerancia a fallos más flexibles, así como mejor integración con los log.

https://www.arsys.es/blog/cron-jobs-una-guia-completa#tree-3

Fichero etc/crontab (debian)


```
user@debian:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the
`crontab' # command to install the new version when you
edit this file
# and files in /etc/cron.d. These files also have username
fields, # that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bi
n

# m h dom mon dow user command
 17 *              * * *   root       cd / && run-parts --report /etc/cron.hourly
 25 6              * * *   root            test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
 47 6              * * 7   root            test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
 52 6 #              1 * *                 test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly
                     root                  )
```

Anacron

Anacron se usa para ejecutar comandos periódicamente, con una frecuencia especificada en días.

Al contrario que cron, no asume que la máquina está corriendo permanentemente.

- se puede usar en máquinas que no están corriendo las 24 horas para controlar tareas periódicas diarias, semanales y/o mensuales que son (eran) usualmente controladas por cron.

Ejemplo cron/anacron

etc/crontab

root@debian78:/var/spool/cron/crontabs# cat /etc/crontab
/etc/crontab: system-wide crontab
Unlike any other crontab you don't have to run the `crontab'
command to install the new version when you edit this file
and files in /etc/cron.d. These files also have username fields,
that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

m h dom mon dow user command
17 *   * * *   root cd / && run-parts --report /etc/cron.hourly
25 6   * * *   root test -x /usr/sbin/anacron || (cd / && run-parts --report /etc/cron.daily)
47 6   * * 7   root test -x /usr/sbin/anacron || (cd / && run-parts --report /etc/cron.weekly)
52 6   1 * *   root test -x /usr/sbin/anacron || (cd / && run-parts --report /etc/cron.monthly)
#

Ejemplo cron/anacron

etc/anacrontab

root@debian78:/var/spool/cron/crontabs# cat /etc/anacrontab # /etc/anacrontab: configuration file for anacron

See anacron(8) and anacrontab(5) for details.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin HOME=/root
LOGNAME=root

1	5	cron.daily	run-parts --report /etc/cron.daily
7	10	cron.weekly	run-parts --report /etc/cron.weekly
@monthly	15	cron.monthly	run-parts --report /etc/cron.monthly

These replace cron's entries

Ejemplo cron/anacron

Contenido del directorio /etc/cron.d

- Fichero /etc/cron.d/anacron:

  root@debian78:/etc# cat /etc/cron.d/anacron

  # /etc/cron.d/anacron: crontab entries for the anacron package

  SHELL=/bin/sh
  PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

  30 7 * * * root test -x /etc/init.d/anacron &&
  /usr/sbin/invoke-rc.d anacron start >/dev/null

Ejemplo cron/anacron

```
root@debian78:/etc# ls /etc/cron.daily
        0anacron apt aptitude bsdmainutils dpkg exim4-base
        logrotate man-db mlocate passwd

root@debian78:/etc# cat /etc/cron.daily/0anacron
        #!/bin/sh
        #
        # anacron's cron script
        #
        # This script updates anacron time stamps. It is called through run-parts
        # either by anacron itself or by cron.
        #
        # The script is called "0anacron" to assure that it will be executed
        # _before_ all other scripts.

        test -x /usr/sbin/anacron || exit 0
        anacron -u cron.daily
```

Ejemplo cron/anacron

root@debian78:/etc# cat /etc/cron.daily/logrotate

```
#!/bin/sh
test -x /usr/sbin/logrotate || exit 0
/usr/sbin/logrotate /etc/logrotate.conf
```

Gracias

